



Documentation technique M2L Mobile

1. Introduction	3
1.1 Objectif et Champ d'Application	3
1.2 Public Visé	3
2. Architecture du Projet	3
2.1 Vue d'Ensemble	3
2.2 Composants et Modules	4
2.3 Flux de Données	4
3. Technologies et Outils Utilisés	4
3.1 Langages et Frameworks	4
3.2 Hébergement et Infrastructure	4
3.3 Outils de Déploiement	4
3.4 Outils de Collaboration	4
4. Environnements	5
4.1 Développement	5
4.2 Test / QA	5
4.3 Production	5
5. Procédure de Déploiement	5
5.1 Pré-requis	5
5.3 Déploiement Manuel	5
6. Maintenance et Mises à Jour	5
6.1 Supervision et Monitoring	6
6.2 Backups et Restauration	6
6.3 Mises à Jour	6
6.4 Gestion des Incidents	6
7. Sécurité	6
7.1 Gestion des Accès	6
7.2 Mesures de Protection	6
7.3 Politique de Sécurité	6
8. Tests et Qualité	6
8.1 Tests Unitaires et d'Intégration	7
8.3 Couverture et Rapports	7
9. Roadmap et Versions	7
9.1 Historique des Versions	7
9.2 Roadmap	7
10. Guide de Contribution	7
10.1 Workflow Git	7
11. Glossaire et Références	7
11.1 Glossaire	7
11.2 Références	8
Conclusion	8

Documentation Technique - Application Mobile M2L

1. Introduction

1.1 Objectif et Champ d'Application

Cette documentation fournit les informations nécessaires pour déployer, maintenir et faire évoluer l'application mobile de réservation de salle de l'entreprise M2L. Elle couvre :

- L'architecture technique
- Les technologies utilisées
- L'infrastructure de déploiement
- Les procédures de maintenance et sécurité

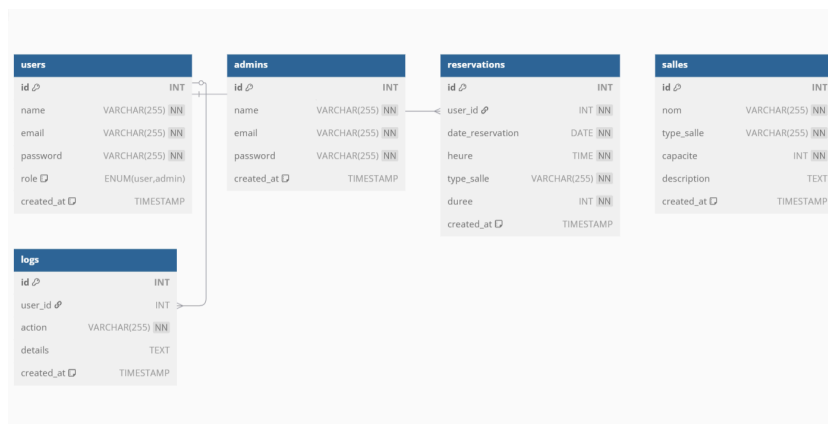
1.2 Public Visé

Destinée aux développeurs front-end, back-end et DevOps impliqués dans la gestion du site web. Un niveau intermédiaire en :

- Développement mobile (Flutter, Dart)
 - Node.js, MongoDB
 - Déploiement d'applications mobiles
-

2. Architecture du Projet

2.1 Vue d'Ensemble



Le site web suit une architecture client-serveur :

- Client : Flutter/Dart
- Back-end : API Node.js (partagée avec le site e-commerce)
- Base de données : MongoDB
- Déploiement : Local via émulateurs Android Studio et iOS

2.2 Composants et Modules

- Front-end : Flutter pour la gestion des rendez-vous
- Back-end : API Node.js pour l'authentification et gestion des données
- Base de données : MySQL
- Tests : Émulateurs Android et iOS

2.3 Flux de Données

- Requêtes API envoyées depuis le site web vers Node.js
 - Traitement des données avec SQL et réponse en JSON
 - Authentification gérée via JWT
-

3. Technologies et Outils Utilisés

3.1 Langages et Frameworks

- Front-end : Flutter (Dart)
- Back-end : Node.js (16.x)
- Bibliothèques principales :
 - Dio (requêtes HTTP)
 - Provider (gestion d'état)
 - Shared Preferences (stockage local)
 - JWT (authentification)

3.2 Hébergement et Infrastructure

- Déploiement : Local uniquement
- Back-end : VM avec API Node.js
- Tests : Émulateurs Android Studio / iOS

3.3 Outils de Déploiement

- Gestionnaires de packages : pub (Flutter) et npm (Node.js)
- VSCode : Pour la compilation du code sur un émulateur

3.4 Outils de Collaboration

- Versioning : GitHub
 - Revue de code : Pull requests obligatoires
-

4. Environnements

4.1 Développement

- Flutter SDK, Android Studio/XCode, Node.js, npm, MongoDB
- Fichier .env pour variables d'environnement
- Commande : flutter run

4.2 Test / QA

- Tests manuels sur émulateurs

4.3 Production

- Déploiement en local
-

5. Procédure de Déploiement

5.1 Pré-requis

- Apache 2.4 installé et configuré sur la VM
- Node.js (16.x) et npm installés
- Base de données SQL installée et accessible

5.3 Déploiement Manuel

1. Cloner le repo : git clone <https://github.com/carterada/M2L-FrontApp>
 2. Aller dans le projet : cd m2l-frontapp
 3. Installer les dépendances : flutter pub get
 4. Configurer le .env
 5. Démarrer le serveur : node server.js
-

6. Maintenance et Mises à Jour

6.1 Supervision et Monitoring

- Logs visibles via console Flutter

6.2 Backups et Restauration

- Code source sauvegardé sur GitHub
- Pas de backup spécifique pour la base de données

6.3 Mises à Jour

- flutter pub upgrade après tests
- flutter clean et flutter pub get après mises à jour majeures

6.4 Gestion des Incidents

- Contact : adminm2l@gmail.com
 - Vérifications :
 - Logs API
 - Émulateurs
 - Connectivité MySQL
-

7. Sécurité

7.1 Gestion des Accès

- Variables sensibles stockées dans .env
- Tokens JWT sécurisés
- Authentification requise pour l'API

7.2 Mesures de Protection

- Validation des entrées
- Protection contre injections SQL

7.3 Politique de Sécurité

- Aucune donnée sensible stockée localement
-

8. Tests et Qualité

8.1 Tests Unitaires et d'Intégration

- Unitaires : Tests des fonctions de l'API avec JestJS
 - Intégration : Tests des flux utilisateur sur le site
-

9. Glossaire et Références

9.1 Glossaire

- API : Interface de communication entre front-end et back-end

- Flutter : Framework multi-plateformes (Google)
- JWT : JSON Web Token
- MySQL : Base de données NoSQL
- Node.js : Environnement JavaScript côté serveur

9.2 Références

- Documentation Flutter, Dart, Node.js, MongoDB, Dio
-

Conclusion

Cette documentation technique évoluera avec le projet. Toute modification majeure de l'architecture ou des processus devra être mise à jour afin d'assurer sa pertinence.